Backdoor Attack on Encryption-Protected Vertical Federated Learning

Junyu Chen, Depeng Chen, Jie Cui, *Senior Member, IEEE*, and Hong Zhong

***Abstract*—Vertical Federated Learning (VFL), as one of the key paradigms in federated learning, is commonly employed in scenarios where multiple parties share the same sample set but possess different features for these samples. Previous studies have demonstrated that VFL is vulnerable to backdoor attacks due to its inherent characteristics. However, the issue of backdoor attacks in encryption-protected VFL has been underexplored. In such scenarios, adversaries cannot directly access plaintext sample-level gradients, which seemingly offers enhanced security for VFL. Adversaries are restricted to leveraging their own bottom model and a small subset of auxiliary samples to conduct backdoor attacks, rendering many existing attack strategies ineffective. In this paper, we propose a powerful backdoor attack: BAEVFL (Backdoor Attack on Encryption-protected Vertical Federated Learning), which is executed through three key stages: pseudo-label inference, trigger optimization, and backdoor poisoning. Our attack can be successfully launched without access to plaintext gradient information or auxiliary samples including all classes. Instead, it requires only the adversary's bottom model and a minimal set of target class samples. We conducted extensive experiments demonstrating that BAEVFL outperforms various state-of-the-art baseline methods, achieving over 98% ASR on four benchmark datasets, while maintaining a utility drop of less than 0.3%. Additionally, we evaluated the effectiveness of current representative defense methods against our BAEVFL. The results indicate that existing defenses fail to strike a balance between defense and utility, and we provide key suggestions for potential improvements to these methods. The BAEVFL, with its stealth and effectiveness, exposes significant security vulnerabilities in encryption-protected VFL, underscoring the urgent need for future research on robust defense mechanisms for this paradigm.**

***Index Terms*—Backdoor attack, vertical federated learning, encryption-protected.**

I. Introduction

FEDERATED learning, a privacy-preserving distributed learning paradigm, provides an effective solution to address the challenges posed by data silos. It is commonly categorized into three types based on the nature of the data held by the participants: Horizontal Federated Learning (HFL) [1], Vertical Federated Learning (VFL) [2], and Federated Transfer Learning (FTL) [3]. The VFL [2] is particularly suitable for collaborative training between entities from different industries. For instance, a bank may collaborate with an E-commerce platform to develop a model for loan risk prediction. Recently, split neural network frameworks [4] have emerged as the predominant implementation method for VFL. Hence, this paper adopts such a framework as the basis for the VFL paradigm. Typically, this framework consists of two roles: the active party and the passive party. The active party initiates the collaborative learning task, possessing a subset of the features for all samples, along with the labels corresponding to the task. It typically operates both a bottom and a top model and seeks to incorporate additional features from the passive party to enhance the quality of its services. The passive party, holding only a subset of the features for all samples, participates in the training process using the bottom model he holds, in exchange for potential benefits.

Previous studies have demonstrated that VFL is vulnerable to backdoor attacks initiated by malicious passive parties [5], [6], [7], [8], [9], [10]. However, these works either depend on access to plaintext, sample-level gradients to infer additional target class samples [5], [6] or optimize triggers [8], or they make overly strong assumptions about the availability of auxiliary data for the attacker [7], [9], [10]. Specifically, they assume that the adversary possesses a specific quantity of data from each class [9], [10] or holds a significant number of samples from the target class (such as 500 samples) [7]. In practice, acquiring such a large volume of auxiliary data for attacks is often impractical, which contradicts the very definition of the VFL paradigm. Moreover, as noted in [2], [11], and [28], an active party can effectively mitigate the attack by simply using encryption techniques like homomorphic encryption and secure multi-party computation. These methods ensure that sample-level plaintext gradients remain inaccessible to passive participants during the learning task. As a result, the adversary can not use sample-level gradients to infer additional target class samples or optimize triggers to aid their attack. Therefore, prior research does not pose a sufficient threat to the security of VFL, nor does it reveal the more severe privacy and security risks inherent to this learning paradigm. In light of this, our work focuses on backdoor attacks within the context of encryption-protected VFL. We propose novel attack method that exposes critical privacy and security vulnerabilities in current VFL paradigms, stemming from their inherent characteristics.

Received 27 January 2025; revised 19 May 2025; accepted 4 June 2025. Date of publication 18 June 2025; date of current version 14 July 2025. This work was supported in part by the National Natural Science Foundation of China under Grant U23A20308 and Grant 62202008; in part by the Natural Science Foundation of Anhui Province, China, under Grant 2408085JX010; and in part by the University Synergy Innovation Program of Anhui Province under Grant GXXT-2022-049. The associate editor coordinating the review of this article and approving it for publication was Prof. Guowen Xu. *(Corresponding authors: Depeng Chen; Jie Cui.)*

The authors are with the Key Laboratory of Intelligent Computing and Signal Processing of Ministry of Education, School of Computer Science and Technology, Anhui University, Hefei 230601, China (e-mail: e23301285@stu.ahu.edu.cn; depengchen@ahu.edu.cn; cuijie@mail.ustc.edu.cn; zhongh@ahu.edu.cn).

Digital Object Identifier 10.1109/TIFS.2025.3581095

1556-6021 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Anhui University. Downloaded on September 07,2026 at 00:37:36 UTC from IEEE Xplore. Restrictions apply.

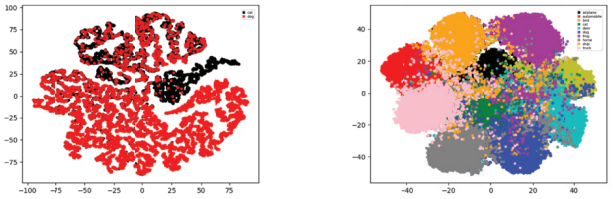


Fig. 1. t-SNE projection of the outputs of the adversary's bottom model. Different color represents different labels. The left figure was plotted on the Dog vs. Cat dataset, while the right figure was plotted on the CIFAR-10 dataset. The embeddings of samples from different classes exhibit clear boundaries between them.

It is important to note that **the attack scenario we explore differs from those addressed in previous related works** in two key aspects: (1) The adversary cannot obtain plaintext, sample-level gradients; (2) The adversary neither possesses a specific quantity of samples from all classes nor has access to a large number of target class samples. In this attack scenario, the adversary can only leverage encrypted gradients, their own bottom model, and a very limited number of target class samples. A small number of target class samples, as noted in [12], can be obtained through purchasing channels. In our experiments, we assume possession of 4 target class samples for datasets such as Dog vs. Cat, CIFAR-10, and CINIC-10, and 10 target class samples for CIFAR-100. Consequently, backdoor attacks in this context face three major challenges: (1) Without access to a large quantity of auxiliary samples or the ability to infer additional target class samples from plaintext, sample-level gradients, **how can the adversary identify target class samples and distinguish them from non target class samples?** (2) In the absence of plaintext, sample-level gradients, **how can the adversary optimize the triggers necessary for a successful backdoor attack?** (3) **How can the adversary perform more comprehensive and effective poisoning to adapt to more complex multi-class classification tasks?**

To address the aforementioned challenges, we propose BAEVFL, a three-stage backdoor attack framework consisting of pseudo-label inference, trigger optimization, and backdoor poisoning. **Each of these stages is designed to overcome the specific challenges outlined above.** The intuition behind our attack method is that a well-trained bottom model should generate distinct embeddings for samples from different classes, as shown in Fig. 1, with **clear boundaries** between them. Based on this, unsupervised clustering algorithms can be employed to group sample embeddings, where the number of clusters is set to match the number of classes in the classification task, thereby establishing a one-to-one correspondence between cluster labels and true class labels. In the pseudo-label inference stage, unsupervised clustering algorithms and a small set of target class samples are utilized to determine the pseudo-labels for all samples, as well as the corresponding pseudo-labels for the target class. During the trigger optimization stage, a surrogate top model is trained using the constructed pseudo-labeled embedding dataset. This surrogate top model is then integrated with the adversary's bottom model to form an inference model used for trigger optimization. In the backdoor poisoning stage, we apply poisoning to a subset of samples from each class, replacing the gradients of poisoned non target samples with those of clean target class samples. This approach enables a more comprehensive and effective poisoning strategy, making the attack robust against real-world multi-class tasks.

In summary, the main contributions of this paper are as follows:

(1) To the best of our knowledge, this work is the first to explore backdoor attacks in encryption-protected VFL scenarios based on split neural networks, introducing a robust backdoor attack: BAEVFL. Unlike previous backdoor attacks in VFL, BAEVFL requires only a small number of target class samples (obtained through internal purchase channels) and the adversary's local bottom model.

(2) We introduce a pseudo-label inference mechanism in BAEVFL, offering a novel solution to the challenge of determining the labels of all samples—a critical requirement for executing effective backdoor attacks.

(3) We offer a novel solution for efficiently optimizing backdoor triggers in encryption-protected VFL scenarios, where adversaries lack access to plaintext sample-level gradients. Additionally, we propose a comprehensive poisoning strategy that enables BAEVFL to effectively adapt to complex multi-class tasks.

(4) We conducted extensive comparative experiments to demonstrate the superiority of BAEVFL. Furthermore, we performed comprehensive ablation studies to identify factors influencing BAEVFL's performance. Finally, by evaluating the effectiveness of common defense mechanisms against BAEVFL, we show that existing defenses are ineffective against this powerful attack and provide several recommendations for improving these defense strategies.

II. Background

In this section, we first introduce VFL based on the split neural network framework. Then, we present an overview of encryption-protected VFL, followed by an analysis of the challenges in executing backdoor attacks within such realistic adversarial contexts.

A. VFL Based on Split Neural Network

VFL [2] is particularly well-suited for collaborative learning among commercial entities from different sectors. These entities often possess datasets with many overlapping samples, but their feature spaces are distinct and complementary. In recent years, the split neural network framework [4] has emerged as the dominant architecture for implementing the VFL paradigm.

Fig. 2 illustrates a VFL scenario involving two participating entities, as discussed in [12], [13], [14], [15], and [16], which represent common real-world applications of VFL. Typically, the initiator of the learning task is referred to as the active party (denoted as party b), who holds the task-related label set $y = \{y_1,...,y_n\}$ and a partial feature set $X^b = \{x_1^b,...,x_n^b\}$, where n is the num of overlapping samples between parties after Private Set Intersection (PSI). The other participant, supporting the active party in the learning process for potential benefit, is called the passive party (denoted as party a). The passive

Authorized licensed use limited to: Anhui University. Downloaded on September 07,2026 at 00:37:36 UTC from IEEE Xplore. Restrictions apply.

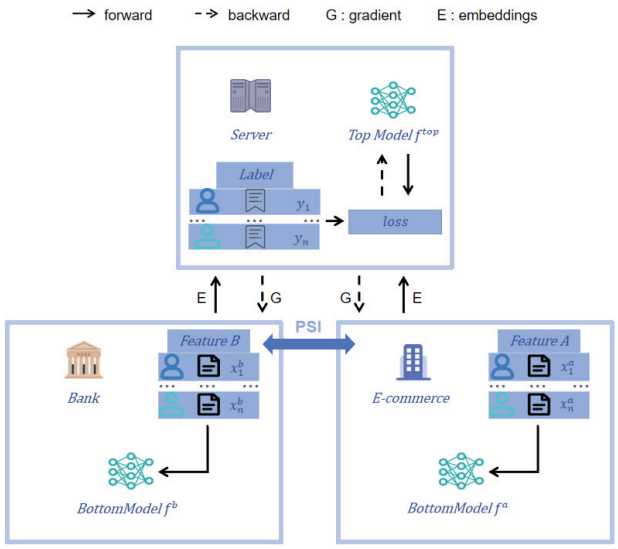


Fig. 2. The VFL architecture based on split neural network: the typical framework of vertical federated learning generally consists of the active party and the passive party.

party holds a dataset with only a partial feature set $X^a = \{x_1^a,...,x_n^a\}$. During each training round, both the passive and active parties feed their respective feature sets X^a and X^b into their respective bottom models to obtain embeddings $E^a = \theta(f^a)(X^a = \{x_1^a, \ldots, x_n^a\})$ and $E^b = \theta(f^b)(X^b = \{x_1^b, \ldots, x_n^b\})$, where $E^a = \{E_1^a,...,E_n^a\}$ and $E^b = \{E_1^b,...,E_n^b\}$, θ denotes the parameters of model. The passive party sends its embeddings E^a to the active party, which then aggregates E^a and E^b, and forwards the aggregated embeddings to the top model f^{top} for forward propagation, *loss* computation, and subsequent backpropagation. After backpropagation, the active party updates its top model and sends the gradient $\frac{\partial \text{ loss}}{\partial E^a}$ and $\frac{\partial \text{ loss}}{\partial E^b}$ of the *loss* with respect to the bottom model embeddings E^b and E^a back to passive party and itself, respectively. Both bottom models are then updated, marking the end of the training round. This iterative process continues for multiple rounds until the VFL model converges.

The aforementioned framework can be easily extended to VFL scenarios involving multiple participants (i.e., more than two passive parties). However, given that real-world VFL scenarios typically consist of only one active party and one passive party, our work primarily focuses on the two-party case. In practice, our proposed BAEVFL can also be seamlessly extended to multi-party settings. We demonstrate this flexibility in our evaluation experiments, where we apply BAEVFL to scenarios with multiple participants. The experimental results confirm that BAEVFL effectively adapts to VFL settings with multiple parties, maintaining strong performance.

B. Encryption-Protected VFL

Recent studies have shown that the split neural network-based VFL paradigm is vulnerable to label inference attacks [12], [15], backdoor attacks [5], [6], [8], and gradient inversion attacks [27], primarily due to the direct exposure of plaintext, sample-level gradients. To protect privacy in the VFL process, cryptographic mechanisms such as homomorphic encryption and secure multi-party computation have been integrated into VFL [2], [11]. While these techniques may introduce additional computational overhead, advancements in this field [17], [18] are gradually alleviating this issue and are being increasingly adopted in real-world frameworks such as FATE [26]. In the encryption-protected VFL scenario based on split neural networks, the passive party only has access to ciphertext gradients $\left[\left[\frac{\partial \text{ loss}}{\partial E^a}\right]\right]$, where [[.]] denotes homomorphic encryption.

Thus, in this scenario, the passive party cannot leverage the sample-level gradients provided by the active party to aid in a backdoor attack. However, to ensure that the bottom models of each collaborating party are normally updated, the encrypted sample-level gradients should be accessible to each collaborator.

TABLE I
SYMBOLS AND NOTATIONS

Notations	Definition
i	pseudo-label
h	the number of classes in this vertical federated learning classification task
H_i	$H_i(i = 0, 1, ..., h-1)$ denotes a list consisting of the IDs of all sample with pseudo-label i and the corresponding their embedding vectors
Y_target_pseudo	the pseudo-label corresponding to the target class
H_{target}	H_{target}=H_i(when i = Y_target_pseudo)
num_poison	the total number of poisoned samples, obtained by multiplying the poisoning rate by the training dataset size
num_target	the size of D_target, it is a hyperparameter specified by the adversary
D_target	target class sample set, it is used for gradient replacement during the poisoning process
D_poison_target	poisoned target class dataset, with a size of num_poison/h
$D_poison_untarget$	poisoned non target class dataset, with a size of $(num_poison/h) \times (h-1)$
D_infer_train	inference head training set, the features in this dataset are the embeddings of the training samples, while the labels are the pseudo-labels assigned to each embedding

III. THE BAEVFL ATTACK

In this section, we first define the threat model that underlies the attack. We then provide an overview of the BAEVFL framework, including a detailed explanation of the three stages involved in BAEVFL. The detailed explanation of the symbols used in follow section provided in Table I.

A. Threat Model

1) Adversary's Goal: In our work, we assume the adversary to be a malicious passive party, as it would be counterproductive and against the interests of the active party to engage in a backdoor attack themselves. For their own benefit, a passive adversary may inject a backdoor into the VFL model, causing the model to predict samples embedded with the trigger as a

Authorized licensed use limited to: Anhui University. Downloaded on September 07,2026 at 00:37:36 UTC from IEEE Xplore. Restrictions apply.

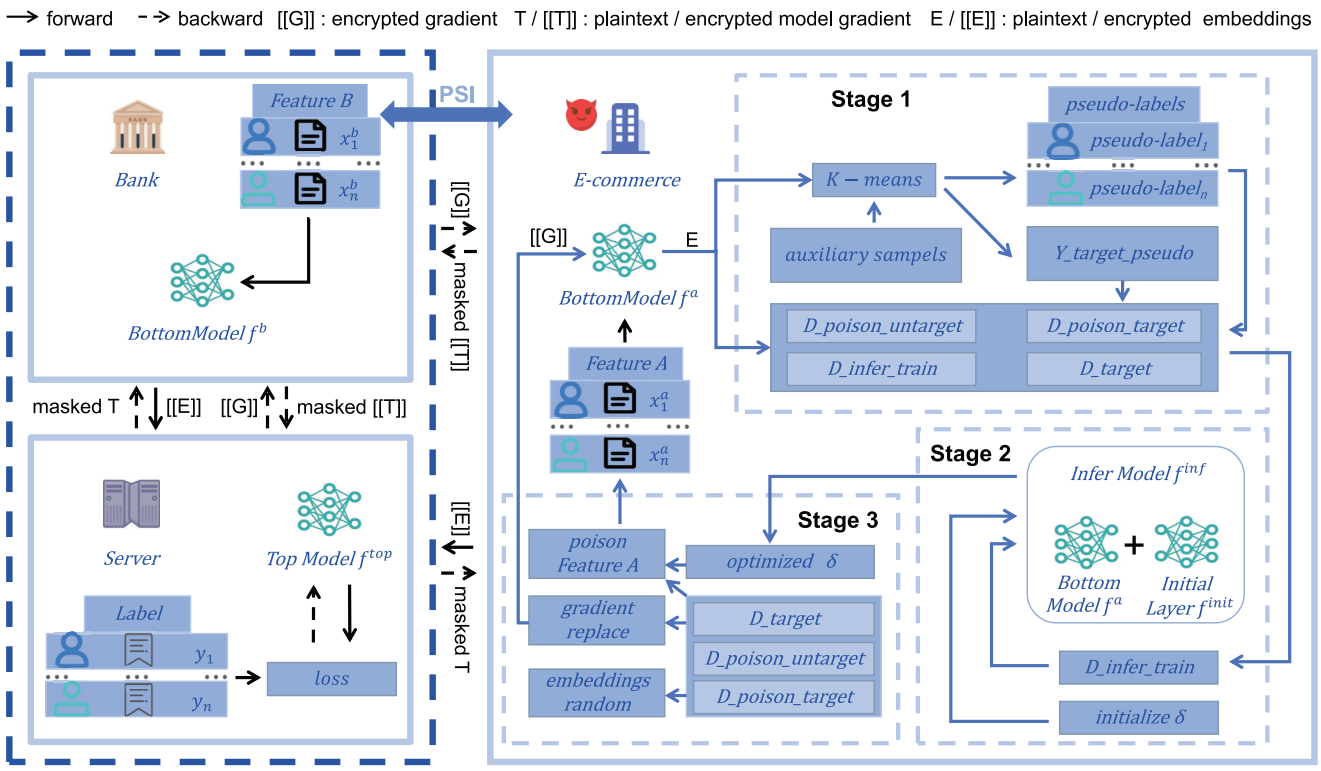


Fig. 3. The overview of the BAEVFL: When the adversary reaches the *infer_epoch*, he or she implement stage1: pseudo-label inference, to determine the pseudo-labels corresponding to the target class and the four datasets required for the attack. The adversary then proceeds to the stage2: trigger optimization, where an Initial Layer is trained, an inference model is constructed, and the trigger is optimized. Upon reaching the *poison_epoch*, the adversary executes the stage3: backdoor poisoning, using the optimized trigger to poison the model and implant the backdoor.

pre-defined target label. For instance, in a scenario where an online E-commerce platform collaborates with a bank to build a loan risk prediction model, the E-commerce platform could plant a backdoor. During the prediction phase, it could inject triggers into the transaction history of certain low-credit users, causing the model to misclassify them as high-credit users, leading to erroneous loan approvals and ultimately boosting the platform's transaction volume.

2) Adversary's Capabilities: As outlined in Section II, the exposure of plaintext, sample-level gradients can lead to significant privacy and security risks. Therefore, homomorphic encryption and secure multi-party computation (SMPC) are commonly employed by the active party to protect these gradient values. In our work, we assume the adversary has only access to the following: (1) Encrypted sample-level gradients, which are necessary for performing backpropagation on the local model. (2) A very limited number of target class samples (in our experiments, 4 target samples for Dog vs. Cat, CIFAR-10, and CINIC-10, and 10 target samples for CIFAR-100). (3) The adversary's bottom model.

B. Overview of the BAEVFL Framework

The framework of BAEVFL is illustrated in Fig. 3. The BAEVFL scheme consists of three stages: Stage 1: pseudo-label inference, Stage 2: trigger optimization, and Stage 3: backdoor poisoning. Each of these stages addresses a distinct challenge faced by backdoor poisoning attacks in encryption-protected VFL settings: (1) distinguishing target class samples from those of non target classes; (2) optimizing a sufficiently strong trigger that can effectively induce the model to classify poisoned samples as the target class, while minimizing the impact on the model's accuracy when predicting clean samples; and (3) designing a more comprehensive and efficient poisoning strategy to ensure that the backdoor attack can easily adapt to complex multi-class tasks.

1) Overview of the Pseudo-Label Inference: To execute a backdoor attack, the adversary needs to infer sample labels before applying strategic poisoning, a challenge extensively studied in prior research [5], [6], [9], [10]. Existing label inference methods fall into two categories: (1) Inferring target class samples using a few known target class samples and plaintext gradients at the sample level, which identifies whether a sample belongs to the target class [5], [6]; (2) Training a surrogate top model with sufficient samples from each class to predict labels across all classes [9], [10].

In encryption-protected VFL, plaintext gradients are inaccessible, rendering the first method [5], [6] inapplicable. The second method [9], [10] is based on an assumption of auxiliary knowledge that is overly strong. Thus, we follow the first method's assumption [5], [6], where the adversary possesses only a small number of target class samples, potentially obtained through insider trading, without access to non target class samples. The absence of plaintext gradients complicates label inference. Inspired by [12], we focus on the adversary's local bottom model to extract private label information. As the bottom model learns high-dimensional representations, samples from different classes become well-separated.

Authorized licensed use limited to: Anhui University. Downloaded on September 07,2026 at 00:37:36 UTC from IEEE Xplore. Restrictions apply.

The adversary can cluster these embeddings, aligning the number of clusters with the class counts, to assign a pseudo-label (i.e., cluster label) to each sample. It then uses the mean embedding of a limited number of auxiliary samples from the target class to match the target label to its corresponding target pseudo-label.

Our work, center on backdoor attacks rather than dedicated label inference, requires determining: (1) which samples belong to the target class, (2) which belong to non target classes, and (3) whether non target samples can be grouped into their respective classes. While prior methods [5], [6] addressed the first two requirements, our pseudo-label inference meets all three, distinguishing between non target classes rather than treating them as a single group.

2) Overview of the Trigger Optimization: Unlike fixed pixel-pattern triggers (e.g., 3×3 blocks) used in traditional backdoor attacks [21], [22], [23], the goal here is to obtain an optimal perturbation that: (1) effectively captures essential features of the target class, enabling any clean sample with the trigger to be misclassified as the target class, and (2) minimizes performance degradation on clean samples caused by the trigger's poisoning effect.

In HFL scenarios, these objectives can be met by optimizing the trigger on the global model received from the central server. However, in encryption-protected VFL, the adversary only has access to their portion of the bottom model, with the top model and other participants' bottom models unknown. Moreover, the encryption of sample-level gradients prevents direct optimization using the gradients of target class samples [11].

To address this, we propose training a surrogate model for the top model using the Initial Layer training set D_infer_train, generated during pseudo-label inference. Since the adversary establishes a one-to-one correspondence between true labels and pseudo-labels, D_infer_train, created by assigning pseudo-labels to sample embeddings. The surrogate model trained on this dataset approximates the top model's functionality, enabling effective trigger optimization. After training the surrogate model, the adversary concatenates their bottom model with the surrogate model to form a complete inference model capable of predicting labels based on their features. Using this inference model and all training samples, the adversary optimizes the perturbation δ by minimizing the loss with respect to the target pseudo-labels Y_target_pseudo.

3) Overview of the Backdoor Poisoning: After completing the first two phases, the adversary embeds the backdoor into the VFL framework during the poisoning epoch.

Prior poisoning-based VFL backdoor methods fall into two categories: (1) poisoning only target class samples [5], [6], [9], creating a trigger-target label association, and (2) poisoning randomly selected samples [8], including both target and non target classes, using gradient replacement to link the trigger with the target label. The first approach [5], [6], [9] faces key limitations: (1) the model lacks exposure to non target class samples with the trigger, reducing backdoor efficacy in multi-class tasks, and (2) limited samples per class in multi-class datasets, such as CIFAR-100, where each class has only 500 samples, leading to insufficient poisoned data if the poisoning rate exceeds 1%. To address these challenges, our work follows the second approach [8], leveraging broader poisoning strategies to enhance backdoor effectiveness.

Previous work [8] using random sample poisoning in each training round suffered from high randomness and uneven class distribution of poisoned samples. In contrast, our approach strategically poisons samples from two datasets generated during pseudo-label inference: the target class poisoning set D_poison_target and the non target class poisoning set $D_poison_untarget$, ensuring a balanced distribution across all classes. This dual poisoning strategy serves two purposes: 1) Poisoning target class samples directly links the trigger to the target label, and 2) Poisoning non target class samples, combined with gradient replacement, indirectly reinforces the trigger-target label association, improving the model's generalization of the backdoor.

After poisoning samples in D_poison_target and $D_poison_untarget$ with the trigger δ, we replace the embeddings of poisoned samples with randomly generated embeddings before sending them to the active party. This prevents the original features of samples from interfering with the trigger-target label association [24]. Upon receiving encrypted gradients from the active party, the adversary replaces the encrypted gradients of non target class poisoned samples with those of target class samples from D_target. This method effectively substitutes changing the non target class poisoned samples' labels to the target label. The rationale is that, after applying the optimized trigger δ, poisoned non target class samples should closely resemble target class samples, allowing the gradient of the loss with respect to the target label for target class samples approximate that of poisoned non target class samples with respect to the same target label. During backpropagation, the adversary uses the original, non-randomized embeddings for the non target class poisoned samples.

IV. Methodology

In this section, we provide a detailed description of the three stages involved in BAEVFL: Stage 1: pseudo-label inference, Stage 2: trigger optimization, and Stage 3: backdoor poisoning. The overall algorithm of BAEVFL in the common two-party encryption-protected VFL scenario is presented in Alg. 1.

A. Pseudo-Label Inference

When the $infer_epoch$ is reached, the adversary initiates the pseudo-label inference by applying a K-means clustering algorithm, where the num of clusters is set to match the num of classes in the learning task. This establishes a one-to-one correspondence between the true labels and pseudo-labels (the cluster labels). The adversary then calculates the mean embedding of auxiliary data (i.e., the small number of target class samples held by the adversary) and feeds this embedding mean into the clustered groups to identify the target pseudo-label corresponding to the target label. Then the adversary can generate four auxiliary datasets for subsequent stages of the attack: (1) a poisoned target class dataset D_poison_target, (2) a poisoned non target class dataset $D_poison_untarget$, (3) an Initial Layer training set D_infer_train, (4) a target class dataset D_target. These datasets are critical for successfully carrying out the remaining two stages of BAEVFL.

Authorized licensed use limited to: Anhui University. Downloaded on September 07,2026 at 00:37:36 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1 The BAEVFL Process in Two-Party Setting

Input: infer epoch: $infer_epoch$, poison epoch: $poison_epoch$, num of VFL training epochs: num_epoch, gradient amplification factor: amp, learning rate of VFL: η, loss function: $loss$, training dataset samples IDS: D
Output: model parameters: $\theta(f^a)$, $\theta(f^b)$, $\theta(f^{top})$
client and server initialize $\theta(f^a)$, $\theta(f^b)$, $\theta(f^{top})$, server creates encryption pairs, send public key to each client
for $j = 1$ to num_epoch **do**
for each batch samples IDS B in D **do**
client B execute:
$E^b = \theta(f^b)(x_i^b) /\!/\ i \in B$
sent $[[E^b]]$ to server
client A execute:
$E^a = \theta(f^a)(x_i^a)$, $E_i^{a'} = \theta(f^a)(x_i^a) /\!/\ i \in B$
if $j \geq poison_epoch$ **then**
$Poison_target = B \cap D_poison_target$
$Poison_untarget = B \cap D_poison_untarget$
for $i \in Poison_target \cup Poison_untarget$ **do**
$E_i^{a'} = \theta(f^a)(x_i^a + \delta)$, E_i^a = random embedding
end for
end if
sent $[[E^a]]$ to server
server execute:
decrypt $[[E^a]]$ and $[[E^b]]$
$L = loss(\theta(f^{top})((E^a, E^b)), y)$
$\theta(f^{top}) = \theta(f^{top}) - \eta \cdot \frac{\partial \mathcal{L}}{\partial \theta(f^{top})}$
sent $[[G^a]] = \left[\left[\frac{\partial \mathcal{L}}{\partial E^a}\right]\right]$ to client A, sent $[[G^b]] = \left[\left[\frac{\partial \mathcal{L}}{\partial E^b}\right]\right]$ to client B
client A execute:
$Target = B \cap D_target$
for $i \in Poison_untarget$ **do**
random choose t from $Target$
$[[G_i^a]] = amp \cdot [[G_t^a]]$
end for
$[[T^a]] = [[G^a]] \cdot \left[\left[\frac{\partial E^{a'}}{\partial \theta(f^a)}\right]\right]$
sent masked $[[T^a]]$ to server
client B execute:
$[[T^b]] = [[G^b]] \cdot \left[\left[\frac{\partial E^b}{\partial \theta(f^b)}\right]\right]$
sent masked $[[T^b]]$ to server
server execute:
decrypt masked $[[T^a]]$ and $[[T^b]]$
sent decrypted results to client A and client B
client A execute:
unmask $[[T^a]]$, $\theta(f^a) = \theta(f^a) - \eta \cdot T^a$
client B execute:
unmask $[[T^b]]$, $\theta(f^b) = \theta(f^b) - \eta \cdot T^b$
end for
client A execute:
if $infer_epoch \leq j < poison_epoch$ **then**
D_1, D_2, D_3, D_4, y = pseudo-label inference
$D_poison_target = D_1$
$D_poison_untarget = D_2$
$D_infer_train = D_3$, $D_target = D_4$
$Y_target_pseudo = y$
δ = trigger optimization
end if
end for

Alg. 2 shows the pseudo-label inference algorithm. Below is a description of the pseudo-label inference pipeline:

(1) The adversary collects the embeddings for all samples in the current round and clusters these embeddings using the K-means algorithm, with the num of clusters set to h. Then, they create h lists $H_i(i = 0, 1, \ldots, h-1)$, where each sample embedding and its corresponding sample ID are stored in the list H_i according to its pseudo-label "i"(i.e., cluster label). Within each list H_i, elements are sorted in ascending order based on the distance of their embeddings to the cluster center. Each list element follows the format (sample ID, sample embedding).
(2) The adversary computes the mean of the embeddings for the auxiliary target class samples. This mean embedding is then fed into the clustered groups to determine the target pseudo-label Y_target_pseudo, and the corresponding target list $H_{target} = H_i(\text{i} = Y_target_pseudo)$.
(3) The adversary calculates total number of poisoned samples: num_poison, based on the predetermined poisoning rate. They also determine the size of the target class sample set num_target based on the num of embeddings contained in the target pseudo-label cluster.
(4) From the target list H_{target}, the adversary selects the first num_poison/h sample IDs to form the target class poisoning set D_poison_target. The next $num_poison/h+1$ to $num_poison/h+num_target+1$ sample IDs are selected to form the target class sample set D_target. For non target list H_i, the adversary selects the first num_poison/h sample IDs from each $H_i(i \neq Y_pseudo)$ to create the non target poisoning set $D_poison_untarget$. Finally, they extract the sample embeddings from all elements in each list H_i(i=0,1...h-1) and assign the corresponding pseudo-label "i" as the label for these embeddings, forming the Initial Layer training set D_infer_train.

B. Trigger Optimization

After completing the pseudo-label inference, the adversary proceeds to implement the trigger optimization. He or she first trains a surrogate top model (referred to as Initial Layer) using the inference head training set D_infer_train. This Initial Layer is then combined with the adversary's bottom model to form a full inference model. With this inference model, the adversary optimizes the trigger δ using all samples in D_infer_train and Y_target_pseudo. The optimization aims to refine δ so that it becomes effective in manipulating the model's predictions without compromising performance on clean samples.

Alg. 3 shows the trigger optimization algorithm. Below is a description of the pipeline for trigger optimization:

(1) The adversary initializes a simple fully connected network, referred to as the Initial Layer, and sets the output dimension of the network to h.
(2) The adversary trains the Initial Layer using the inference head training set D_infer_train. Upon completion of the training, the adversary concatenates their bottom model with the Initial Layer to form the inference model.
(3) The adversary initializes the perturbation δ as a tensor with dimensions consistent with the features of the each

Authorized licensed use limited to: Anhui University. Downloaded on September 07,2026 at 00:37:36 UTC from IEEE Xplore. Restrictions apply.

Algorithm 2 The Process of Pseudo-Label Inference

Input: num of classes in VFL classification task: h, auxiliary data D_{aux}, embeddings of all samples from client A in current epoch: E^a, model parameters: $\theta(f^a)$, total num of poisoned samples: num_poison
Output: D_poison_target, $D_poison_untarget$, D_infer_train, D_target, Y_target_pseudo
$t = num_poison/h$
$E_{aux} = \theta(f^a)(D_{aux})$
$e_{aux} = \text{mean}(E_{aux})$
CLUSTER, Y_target_pseudo = K-means(E^a, e_{aux})
identify num_target base on situation of CLUSTER
for $i = 1$ to h **do**
  $H_i = []$
end for
for each embedding in CLUSTER **do**
  dis = distance of embedding to cluster center
  i = cluster label of embedding
  H_i.append((i, sample ID, dis, embedding))
end for
$D_poison_untarget = []$, $D_infer_train = []$
for $i = 1$ to h **do**
  $H_i' = H_i$
  sort H_i and H_i' in ascending order based on dis
  remove i, embedding and dis of every element in H_i
  remove sample ID and dis of every element in H_i'
  if $i = Y_target_pseudo$ **then**
    $H_{target} = H_i$
    $D_poison_target \leftarrow H_{target}(0, t)$
    $D_target \leftarrow H_{target}(t, t + num_target)$
  else if $i \neq Y_target_pseudo$ **then**
    $D_poison_untarget$.append($H_i(0, t)$)
  end if
  D_infer_train.append(H_i')
end for

samples they possess, setting all elements of this tensor to zero. The adversary then associates δ to all training samples.

(4) The adversary freezes all parameters of the inference model and utilizes the all samples associated with δ to optimize δ using the Projected Gradient Descent (PGD) algorithm [25], with the objective of minimizing the loss of all samples with respect to the target pseudo labels Y_target_pseudo.

C. Backdoor Poisoning

When the $poison_epoch$ round is reached, the adversary begins the backdoor poisoning by injecting the optimized trigger δ into both D_poison_target and $D_poison_untarget$. The embeddings of poisoned samples are replaced with random vectors to prevent the clean features of these samples from interfering with the association between the trigger δ and the target label. It is important to note, however, that during the backpropagation process, the original, non-randomized embeddings of poisoned samples are used for computing gradient of model parameters. Upon receiving the encrypted gradients from the active party, the adversary can replace the encrypted gradients of the non target class poisoned samples with those of the clean target class samples from D_target. This substitutes the effect of a label-flipping attack, as if the non target class poisoned samples had been relabeled to the target label. This process effectively facilitates a stealthy backdoor attack by exploiting the gradient replacement technique.

Algorithm 3 The Process of Trigger Optimization

Input: model parameters: $\theta(f^a)$, Initial Layer training set D_infer_train, num of Initial Layer training epochs: $init_epoch$, num of trigger optimization epochs: $optim_epoch$, learning rate of trigger optimization: α, loss function: $loss$, features of samples belong to client A: X^a, the target pseudo-label: Y_target_pseudo
Output: trigger δ
initialize trigger δ, Initial Layer $\theta(f^{init})$
$\theta(f^{init})$ = train($init_epoch$, $\theta(f^{init})$, D_infer_train)
$\theta(f^{inf})$ = concat($\theta(f^a)$, $\theta(f^{init})$), freeze $\theta(f^{inf})$
for $j = 1$ to $optim_epoch$ **do**
  for each batch B in X^a **do**
    $L = loss(\theta(f^{inf})((X_B^a + \delta), Y_target_pseudo))$
    $\delta = \delta - \alpha \cdot \frac{\partial \mathcal{L}}{\partial \delta}$
  end for
end for

Alg. 1 shows the backdoor poisoning algorithm. Below is the pipeline for each batch during the poisoning round:

(1) The adversary checks whether the current batch contains samples belong to the target class poisoning set D_poison_target, the non target class poisoning set $D_poison_untarget$, and the target class sample set D_target. The samples belong to the D_poison_target in the batch are referred to as $Poison_target$, the samples belong to the $D_poison_untarget$ in the batch are referred to as $Poison_untarget$, the samples belong to the D_target in the batch are referred to as $Target$.

(2) The adversary injects the trigger δ into the samples in both $Poison_target$ and $Poison_untarget$, then performs a standard forward pass through the bottom model to obtain the embeddings for all samples in the batch.

(3) The adversary replaces the embeddings of the samples in both $Poison_target$ and $Poison_untarget$ with randomly generated embeddings and sends the embeddings of the entire batch to the active party.

(4) Upon receiving the sample-level encrypted gradients for the batch from the active party, the adversary replaces the encrypted gradients of the $Poison_untarget$ samples with the amplified encrypted gradients of the $Target$ samples.

V. Evaluation

A. Experiment Setup

1) Dataset: We evaluated our proposed BAEVFL on four real-world benchmark datasets: Dog vs. Cat, CIFAR-10, CIFAR-100, and CINIC-10. These datasets respectively represent binary classification tasks, common classification tasks, large-scale classification tasks, and complex multi-class

Authorized licensed use limited to: Anhui University. Downloaded on September 07,2026 at 00:37:36 UTC from IEEE Xplore. Restrictions apply.

TABLE II
DATASET DETAILS FOR THE EXPERIMENTS

Dataset	TrainSet Size	TestSet Size	Number of Labels	Input Size
Dog VS Cat	20,000	5,000	2	$32 \times 32 \times 3$
CIFAR-10	50,000	10,000	10	$32 \times 32 \times 3$
CINIC-10	180,000	90,000	10	$32 \times 32 \times 3$
CIFAR-100	50,000	10,000	100	$32 \times 32 \times 3$

TABLE III
PARAMETER SETTING OF TARGET MODELS IN OUR EXPERIMENTS

Dataset	Epochs	Batch	Learning Rate	Lr-Decay	Milestones
Dog VS Cat	100	256	1e-1	1e-1	50,85
CIFAR-10	100	128	1e-1	1e-1	50,85
CINIC-10	100	2048	8e-1	1e-1	50,85
CIFAR-100	100	256	1e-1	1e-1	50,85

classification tasks. For the Dog vs. Cat dataset, since it only provides training data, we randomly selected 2,500 samples from each class to form a test set. For the CINIC-10 dataset, consistent with prior work [6], [8], [9], [10], we merged the training and validation sets to create a new training set for model training. As for the CIFAR-10 and CIFAR-100 datasets, the training and test set distributions remained unchanged from the original datasets. Detailed information about the four datasets is presented in Table II.

2) Vertical Federated Learning Settings: The default VFL settings in our experiments involves two participants, each holding half of the features for all samples. One participant, designated as the active party, also holds the sample labels. This configuration represents the most common setup in VFL. For all four datasets, the bottom models of each participant adopt a ResNet20 architecture, while the top model consists of four fully connected layers. The default aggregation method concatenates the output vectors(embeddings) from the bottom models. Both the bottom and top models are optimized using the SGD optimizer with a momentum of 0.9. The learning rate is adjusted via the MultiStepLR scheduler, with the milestones listed in Table III. The model achieves the following top-1 accuracies on the datasets: 81.54% on Dog vs. Cat, 82.95% on CIFAR-10, and 73.79% on CINIC-10. For CIFAR-100, the model achieves a top-5 accuracy of 74.93%. Due to the complexity of CIFAR-100, and in line with previous work, the top-5 accuracy is used as the metric to evaluate model utility. Detailed training parameters for each dataset are provided in Table III.

3) Attack Settings: In our attack evaluation experiments, we assessed our BAEVFL and several baselines on four benchmark datasets. For Dog vs. Cat, CIFAR-10, CINIC-10 and CIFAR-100, we assume the adversary holds target class samples numbering 4, 4, 4, and 10, respectively. During each attack implementation, a simple two-layer fully connected network is initialized as the Initial Layer. The Initial Layer is trained for 3 epochs on Dog vs. Cat, CIFAR-10, and CINIC-10 datasets, and for 10 epochs on the CIFAR-100 dataset. The training batch size is set to 64, with the Adam optimizer and a learning rate of 5e-3. During the backdoor poisoning phase, to prevent the model from overfitting to the backdoor task, we use an amplification factor amp_1 for the first two poisoning rounds and amp_2 for subsequent rounds. Detailed parameters, including the poisoning rate, amplification factor, trigger optimization radius, and the size of the target class dataset D_target, are provided in Table IV.

TABLE IV
THE DETAILED PARAMETERS OF THE ATTACK SETTINGS

Dataset	Poisoning Rate	Amplification Factor	Radius	num_target
Dog VS Cat	0.25%	amp_1:20 amp_2:16	1.0	5000
CIFAR-10	0.1%	amp_1:20 amp_2:16	1.0	2500
CINIC-10	0.1%	amp_1:20 amp_2:16	1.0	9000
CIFAR-100	1%	amp_1:12 amp_2:8	1.0	500

4) Baselines: In our evaluation experiments, we compared BAEVFL with four representative baselines on VFL backdoor attacks. The related baseline works are introduced as follows:

VILLIAN. The VILLIAN [6] consists of two stages: target class sample inference and backdoor poisoning. It injects triggers with a pattern of two positive values followed by two negative values into the embeddings of target class poisoned samples during the poisoning phase. Since target class sample inference in VILLIAN requires access to plaintext gradients at the sample level, we implemented only its backdoor poisoning phase in our reproduction. Additionally, VILLIAN introduces a series of enhancement strategies for backdoor poisoning, all of which were faithfully reproduced in accordance with the original paper.

PGBA. Similar to VILLIAN, PGBA [5] consists of two stages: target class sample inference and backdoor poisoning. It only poisons samples belonging to the target class. During the poisoning phase, it replaces the features of the poisoned samples with those of a randomly selected sample from the same batch firstly, and then injects triggers in the form of a 4×4 white square located at the center of the image into target class samples.

BASL. The BASL [7] comprises two stages: trigger vector construction and backdoor poisoning. The adversary first applies the meanshift algorithm on the embeddings of known target class samples to obtain backdoor embedding vector. The vector are then enhanced using a stealthiness improvement strategy. During the poisoning phase, the enhanced vector replace the embeddings of target class samples to achieve poisoning. When reproducing BASL, we retained its noise-based stealthiness improvement strategy. The added noise follows the original paper's specifications, sampled from a normal distribution with a mean of 0 and a variance of 0.5.

TECB. The TECB [8] comprises two stages: trigger optimization and backdoor poisoning. The trigger optimization stage relies on sample-level plaintext gradients to minimize the loss of poisoned samples on the target label, which is clearly impractical in encryption-protected VFL scenarios. Therefore, in our attack evaluation, for a fair comparison, we assume that the adversary uses unoptimized perturbations (where each element is sampled from a uniform distribution over [0,1]) for the poisoning phase. All other implementation details and hyperparameters follow those in the original paper.

Additionally, we consider a VFL framework without backdoor attacks as another baseline. This serves to contrast the

Authorized licensed use limited to: Anhui University. Downloaded on September 07,2026 at 00:37:36 UTC from IEEE Xplore. Restrictions apply.

TABLE V
ATTACK PERFORMANCE OF BAEVFL COMPARED WITH THE BASELINES

Dataset	Metric	VILLIAN	PGBA	BASL	TECB	BASELINE	BAEVFL
cat vs dog	ASR	21.56 ± 2.91	34.18 ± 7.54	67.90 ± 38.52	71.18 ± 13.87	18.40 ± 1.88	$\mathbf{98.00 \pm 1.30}$
	MTA	80.76 ± 0.25	81.66 ± 0.64	81.58 ± 0.83	81.58 ± 0.29	81.54 ± 0.96	81.43 ± 1.12
cifar-10	ASR	2.12 ± 0.23	6.16 ± 1.02	34.07 ± 9.30	82.83 ± 6.18	1.82 ± 0.16	$\mathbf{99.78 \pm 0.23}$
	MTA	82.88 ± 0.23	82.73 ± 0.31	83.03 ± 0.45	82.95 ± 0.24	82.95 ± 0.34	82.73 ± 0.72
cinic-10	ASR	1.76 ± 0.30	2.64 ± 0.26	24.62 ± 4.93	17.84 ± 24.14	1.86 ± 0.14	$\mathbf{98.79 \pm 1.30}$
	MTA	73.77 ± 0.40	73.56 ± 0.36	73.59 ± 0.11	73.95 ± 0.26	73.79 ± 0.32	73.69 ± 0.39
cifar-100	ASR	2.79 ± 0.06	5.49 ± 0.81	81.80 ± 0.98	1.93 ± 0.36	2.61 ± 0.40	$\mathbf{98.71 \pm 1.73}$
	MTA	74.85 ± 0.28	75.38 ± 0.18	74.65 ± 0.52	75.30 ± 0.09	74.93 ± 0.17	75.35 ± 0.15

impact of backdoor attacks on model utility and overall attack performance.

5) Evaluation Metrics and Evaluation Dataset: Consistent with prior work [10], [12], we use the entire test set to comprehensively evaluate the model's utility (MTA), and utilize all non target class samples from the test set to thoroughly assess the success rate of the backdoor attack (ASR), which is the percentage of all non target class samples in the test set misclassified as the specified target label. For the CIFAR-100 dataset, we adopt top-5 accuracy for ASR calculation due to the task's complexity as [10] illustrated. Following previous studies [8], the target classes for the four datasets are "dog" for Dog vs. Cat, "plane" for CIFAR-10, "plane" for CINIC-10, and "apple" for CIFAR-100.

6) Experimental Environment: The experiments were conducted on our workstation equipped with an Intel(R) Core(TM) i7-10700 CPU @ 2.90GHz and four NVIDIA GeForce RTX 3090 GPU cards.

B. Experimental Results and Analysis

1) Comparison With Baselines: Each experiment was conducted multiple times to obtain the mean and variance of the overall performance, and the results are recorded in Table V. As shown, our BAEVFL achieves the strongest attack performance across all four datasets, with an impact on model utility of no more than 0.3%. This demonstrates the significant threat posed by BAEVFL to encryption-protected VFL in real-world scenarios, including binary classification, common classification tasks, large-scale data classification, and complex multi-classification tasks. Furthermore, the negligible impact on model utility highlights the stealthiness of BAEVFL. The ability of BAEVFL to achieve a high ASR while maintaining nearly unaffected model utility can be attributed to the following three factors: 1) The trigger used in BAEVFL poisoning is optimized to capture the crucial features of the target class; 2) The poisoned samples in BAEVFL cover all classes, leading to a more comprehensive poisoning effect; 3) The extremely low poisoning rate in BAEVFL ensures that the model's learning of the primary task is minimally impacted, thus preserving high model utility.

The performance of VILLIAN is poor across all four datasets, only slightly better than the baseline. This may be attributed to its reliance on sample-level plaintext gradients during the label inference phase, which is infeasible in encryption-protected VFL scenarios. Consequently, VILLIAN can only utilize adversary-known target class samples for poisoning, reducing the number of available poisoning samples. Additionally, the trigger used by VILLIAN is an unoptimized pattern of two positive values followed by two negative values, further contributing to its poor attack performance.

The PGBA also demonstrates poor attack performance on all four datasets. Similar to VILLIAN, PGBA cannot infer additional target class samples for poisoning under encryption-protected VFL. Moreover, the trigger used during the poisoning phase is the unoptimized 4×4 white square trigger similar as [21], leading to its limited attack effectiveness.

The BASL achieved moderate attack performance on the Dog vs. Cat and CIFAR-100 datasets, with success rates of 67.90% and 81.80%, respectively. However, these results are still weaker compared to the attack effectiveness of our BAEVFL. On the CIFAR-10 and CINIC-10 datasets, BASL exhibited poor attack performance. This can likely be attributed to the overly strong noise used in BASL's malicious embedding stealth enhancement strategy, which significantly disrupts the original malicious embeddings and reduces attack effectiveness. As noted in the literature, smaller noise levels increase the similarity among malicious embeddings, making the adversarial behavior easier to detect. Therefore, in our experiments, the noise used was kept consistent with that in the original study.

The TECB showed relatively good attack performance on the Dog vs. Cat and CIFAR-10 datasets, achieving success rates of 71.18% and 82.83%, respectively. However, our BAEVFL outperformed TECB with attack success rates of 98.00% and 99.78% on these two datasets. On the CINIC-10 and CIFAR-100 datasets, TECB failed to deliver effective attacks. The overall weak performance of TECB can be attributed to its reliance on sample-level plaintext gradients during the trigger optimization process. In encryption-protected VFL scenarios, this optimization cannot be performed, resulting in the use of unoptimized triggers during the poisoning process.

2) Contributions of Each Phase of BAEVFL to Overall Attack Performance: BAEVFL consists of three phases: the first phase aims to infer pseudo-labels for samples, the second phase focuses on optimizing a trigger to capture key features of the target class, and the third phase embeds the backdoor into the VFL framework. To evaluate the contributions of trigger optimization and backdoor poisoning to the overall attack performance, we divide BAEVFL into two parts: the first and second phases, and the first and third phases. As shown in the experimental results in Table VI, the proposed trigger optimization demonstrates excellent performance. When optimized triggers are directly used for backdoor attacks,

Authorized licensed use limited to: Anhui University. Downloaded on September 07,2026 at 00:37:36 UTC from IEEE Xplore. Restrictions apply.

TABLE VI
ATTACK PERFORMANCE ON DIFFERENT STAGES OF BAEVFL

Dataset	ASR			MTA		
	Stage 1+2	Stage 1+3	Stage 1+2+3	Stage 1+2	Stage 1+3	Stage 1+2+3
dog vs cat	97.90 ± 0.99	91.01 ± 12.65	$\mathbf{98.00 \pm 1.30}$	81.54 ± 0.96	82.19 ± 0.75	81.43 ± 1.12
cifar-10	93.39 ± 4.53	66.63 ± 47.11	$\mathbf{99.78 \pm 0.23}$	82.95 ± 0.34	79.46 ± 4.94	82.37 ± 0.72
cinic-10	93.53 ± 3.66	33.94 ± 11.67	$\mathbf{98.79 \pm 1.30}$	73.79 ± 0.23	73.72 ± 0.27	73.69 ± 0.39

TABLE VII
ATTACK PERFORMANCE OF BAEVFL UNDER TOP MODELS OF DIFFERENT COMPLEXITIES

Top Model		dog vs cat		cifar-10		cinic-10	
		ASR	MTA	ASR	MTA	ASR	MTA
FCNN2	BASELINE	17.42 ± 1.25	81.70 ± 0.29	2.04 ± 0.11	82.14 ± 0.54	1.89 ± 0.05	73.30 ± 0.62
	BAEVFL	$\mathbf{95.98 \pm 2.40}$	81.42 ± 0.85	$\mathbf{99.99 \pm 0.01}$	82.11 ± 0.35	$\mathbf{98.97 \pm 1.20}$	72.54 ± 1.76
FCNN3	BASELINE	17.78 ± 0.89	81.00 ± 1.01	2.00 ± 0.13	82.69 ± 0.41	1.74 ± 0.01	73.52 ± 0.14
	BAEVFL	$\mathbf{92.48 \pm 4.87}$	81.76 ± 0.18	$\mathbf{99.03 \pm 1.36}$	82.66 ± 0.45	$\mathbf{99.70 \pm 0.39}$	73.41 ± 0.89
FCNN4	BASELINE	18.40 ± 1.88	81.54 ± 0.96	1.82 ± 0.16	82.95 ± 0.34	1.86 ± 0.14	73.79 ± 0.32
	BAEVFL	$\mathbf{98.00 \pm 1.30}$	81.43 ± 1.12	$\mathbf{99.78 \pm 0.23}$	82.73 ± 0.72	$\mathbf{98.79 \pm 1.30}$	73.69 ± 0.39
FCNN5	BASELINE	20.26 ± 1.02	79.88 ± 1.17	1.71 ± 0.09	83.22 ± 0.25	1.90 ± 0.05	73.56 ± 0.13
	BAEVFL	$\mathbf{92.48 \pm 7.44}$	81.64 ± 0.04	$\mathbf{94.40 \pm 4.17}$	82.88 ± 0.16	$\mathbf{96.05 \pm 3.57}$	73.63 ± 0.25
FCNN6	BASELINE	17.40 ± 1.41	80.72 ± 0.93	1.78 ± 0.15	83.08 ± 0.19	1.87 ± 0.14	72.96 ± 0.59
	BAEVFL	$\mathbf{97.37 \pm 0.88}$	81.78 ± 0.22	$\mathbf{96.49 \pm 4.01}$	82.94 ± 0.23	$\mathbf{94.94 \pm 2.83}$	73.44 ± 0.18

the attack success rates on the Dog vs. Cat, CIFAR-10, and CINIC-10 datasets reach 97.90%, 93.39%, and 93.53%, respectively. Furthermore, employing these optimized triggers for poisoning further enhances the backdoor attack performance. However, using unoptimized triggers for poisoning alone yields insignificant and unstable results on the three datasets. This instability is likely due to the fact that each element of the unoptimized trigger is randomly sampled from a uniform distribution over [0,1]. Such unoptimized triggers fail to capture the critical features of the target class, leading to attack performance that heavily depends on the trigger's initialization. Consequently, the attack becomes less stable and less effective.

3) Attack Performance of BAEVFL Under Top Models of Different Complexities: In real-world VFL scenarios, the architecture of the top model may vary in complexity, being either more complex or simpler than a four-layer fully connected network. To verify the applicability of BAEVFL across top models of varying complexities, we conducted experiments by setting the top model architecture to 2, 3, 4, 5, and 6 fully connected layers. As shown in the experimental results in Table VII, BAEVFL achieves excellent and stable attack performance across top models of different complexities while exerting only a minimal impact on model utility. This further validates the robustness of the proposed BAEVFL method.

4) Attack Performance of BAEVFL in Multi-Participant Settings: Although VFL architectures typically involve two participants, we extended our evaluation to scenarios with multiple participants to verify the scalability of BAEVFL. Specifically, we increased the number of collaborators to 3, 4, and 5 while keeping the number of malicious collaborators fixed at 1. As shown in the experimental results in Fig. 4, on the CIFAR-10 dataset, an increase in the number of collaborators leads to a decrease in the success rate of backdoor attacks. However, the attack still achieves a notable success rate (e.g., 67.76% when the number of collaborators is 5). The decline in attack success rate can be attributed to the reduced proportion of the malicious participant within the overall VFL framework as the number of collaborators increases, thereby diminishing the influence of malicious embeddings injected by the adversary. In contrast, on the Dog vs. Cat dataset, the attack success rate fluctuates only slightly as the number of collaborators increases. This highlights the vulnerability of binary classification tasks to backdoor attacks and underscores the significant threat posed by BAEVFL to the security of real-world binary classification tasks.

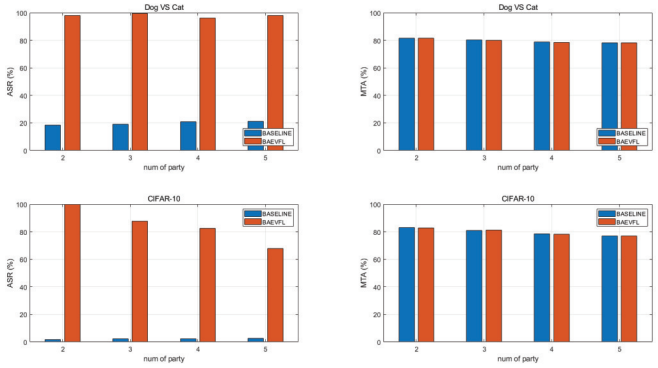


Fig. 4. Attack performance of BAEVFL in multi-participant settings, each collaborating party shares a subset of the total features. Although the ASR of BAEVFL decreases with the increasing number of participants, it still poses a non-negligible threat.

VI. SENSITIVE ANALYSIS

In this section, we further evaluate the factors affecting the attack performance of the BAEVFL and analyze how these factors influence it. The aim is to explore the sensitivity of BAEVFL to poisoning rate, amplification factor, trigger optimization radius, the size of D_target, num of layers in the Initial Layer and the aggregation method in VFL. The sensitivity analysis follows the same experimental settings as the comparative experiments. To simulate three potential

Authorized licensed use limited to: Anhui University. Downloaded on September 07,2026 at 00:37:36 UTC from IEEE Xplore. Restrictions apply.

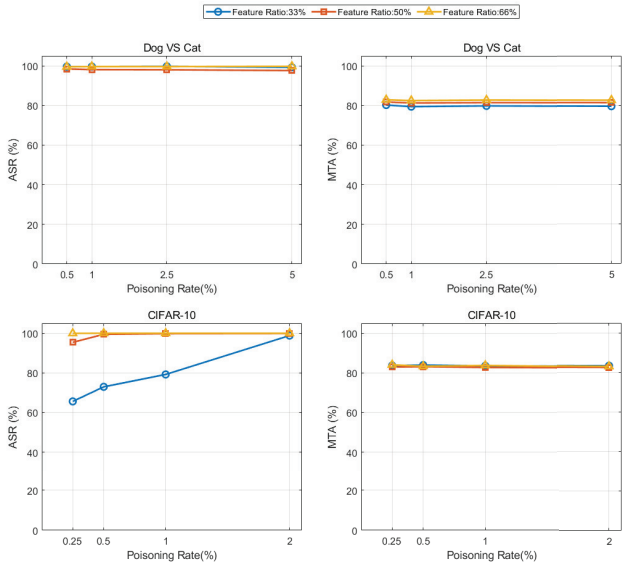


Fig. 5. Attack performance of BAEVFL under different poisoning rate. When the adversary possesses fewer features, a smaller poisoning rate may weaken the ASR.

scenarios—where the malicious passive party holds fewer features than the active party, holds a comparable features, or holds more features—we set the adversary's feature proportion to 33%, 50%, and 66% for each sensitivity analysis experiment, and the results for all three scenarios are presented.

A. Poisoning Rate

The poisoning rate is defined as the percentage of poisoned training samples relative to the total number of training samples. A higher poisoning rate often indicates stronger intervention by the adversary during training. We conducted experiments by setting the poisoning rate to "0.5%, 1%, 2.5%, 5%" for the Dog vs. Cat dataset and "0.25%, 0.5%, 1%, 2%" for the CIFAR-10 dataset. As shown in the experimental results in Fig. 5, for the Dog vs. Cat dataset, changes in the poisoning rate have minimal impact on both attack success rate and model utility. However, for the CIFAR-10 dataset, when the adversary holds a smaller proportion of features (33%), a lower poisoning rate may lead to a decline in attack success rate. This is likely because, in scenarios where the adversary holds fewer features, the model relies more heavily on the benign features of the active party, and a low poisoning rate may fail to embed a sufficiently robust backdoor into the model. Thus, adversaries holding a smaller proportion of features may increase the poisoning rate to enhance the effectiveness of backdoor attacks. Inspired by the findings from the poisoning rate sensitivity analysis, we decided to specify appropriate poisoning rates for the three scenarios with different adversary feature proportions in subsequent sensitivity analyses, as shown in Table VIII.

B. Amplification Factor

The amplification factor influences the learning rate of the backdoor task. A larger amplification factor typically leads to a higher learning rate for the backdoor task, thereby accelerating its convergence during the model training process. To evaluate its impact, we adjusted the amplification factor to "0.25x, 0.5x, 1x, 2x" of the original value for experiments on the Dog vs. Cat and CIFAR-10 datasets. As shown in Fig. 6, for the Dog vs. Cat dataset, increasing or decreasing the amplification factor does not significantly affect backdoor attack performance. However, for the CIFAR-10 dataset, when the adversary holds a smaller proportion of features, a smaller amplification factor may cause the backdoor task to converge more slowly, resulting in poor attack performance. In such cases, the adversary may select a larger amplification factor to accelerate the learning of the backdoor task and maintain robust backdoor attack performance.

TABLE VIII
THE POISONING RATES SPECIFIED FOR THREE SCENARIOS ACROSS THE TWO DATASETS

Dataset	Feature Ratio		
	33%	50%	66%
Dog VS Cat	0.25%	0.25%	0.25%
Cifar-10	0.2%	0.1%	0.05%

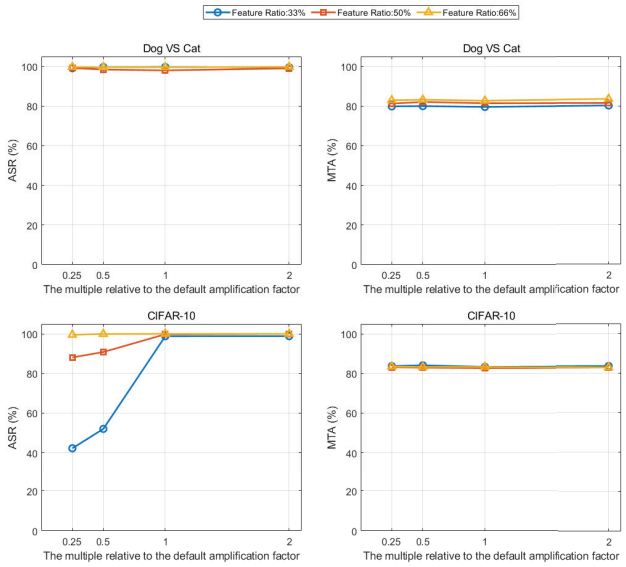


Fig. 6. Attack performance of BAEVFL under different amplification factor. When the adversary possesses fewer features, a smaller gradient amplification factor may weaken the ASR.

C. Trigger Optimization Radius

In BAEVFL, the trigger is additive noise with the same dimensionality as the sample features. As we use the PGD algorithm to optimize the trigger, any element in the trigger tensor exceeding the optimization radius during the process is projected back onto the radius sphere. A larger optimization radius generally provides a broader search space, while a smaller radius limits the search space. To investigate the impact of the trigger optimization radius on BAEVFL, we set the radius to "0.25, 0.5, 1, 2" and conducted experiments. Fig. 7 illustrates the results, showing that when the adversary holds a larger proportion of features, the optimization radius has minimal impact on BAEVFL performance. However, when the adversary holds fewer features, a larger radius may cause

Authorized licensed use limited to: Anhui University. Downloaded on September 07,2026 at 00:37:36 UTC from IEEE Xplore. Restrictions apply.

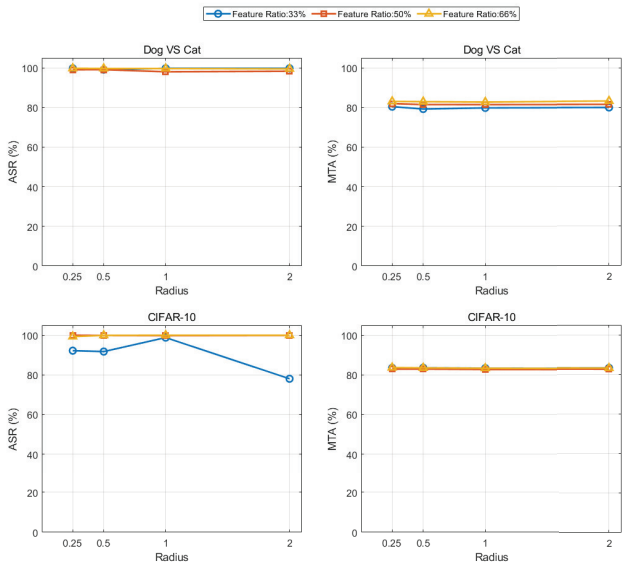


Fig. 7. Attack performance of BAEVFL under different trigger optimization radius. When the adversary possesses fewer features, both larger and smaller trigger optimization radius may influence the ASR of BAEVFL to some extent.

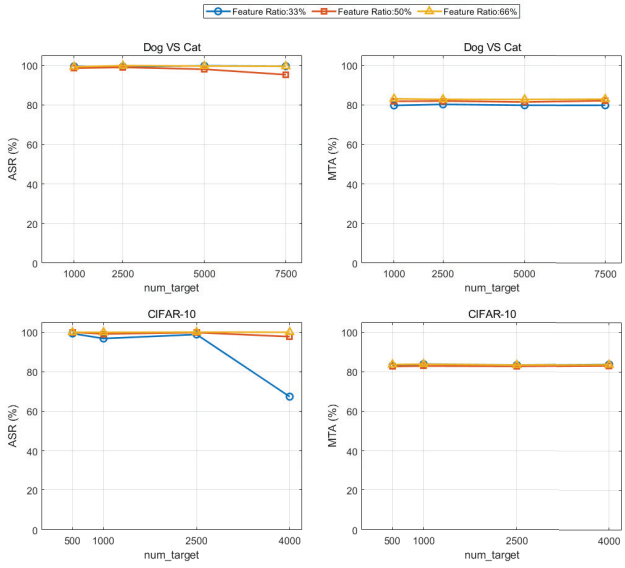


Fig. 8. Attack performance of BAEVFL under different *num_target*. When the adversary possesses fewer features, a larger *num_target* may interfere with backdoor poisoning, thereby weakening the ASR of BAEVFL.

the optimized trigger used for poisoning to deviate from the feature distribution of normal samples, resulting in suboptimal backdoor attack performance. Conversely, a smaller optimization radius restricts the search space, leading to trigger that fail to capture the critical features of the target class, thereby weaking the attack success rate.

D. The Size of D_target (num_target)

In the pseudo-label inference phase of BAEVFL, the adversary construct the target class dataset *D_target* based on clustering results. This dataset is subsequently used to replace the gradients of non target class poisoned samples during the poisoning phase. A smaller size of *D_target* (*num_target*) means fewer target class samples can be identified per batch, potentially resulting in cases where no suitable target class samples are available for gradient replacement. Conversely, a larger *num_target* may include a higher proportion of non target class samples, thereby diminishing the effectiveness of gradient replacement. To evaluate this factor, we set *num_target* to "1000, 2500, 5000, 7500" for the Dog vs. Cat dataset and "500, 1000, 2500, 4000" for the CIFAR-10 dataset.

The experimental results in Fig. 8 show that for the Dog vs. Cat dataset, *num_target* has a slight but negligible impact on BAEVFL performance. However, for the CIFAR-10 dataset, a smaller *num_target* does not reduce the attack success rate. On the other hand, a larger *num_target* leads to a decline in attack success rates when the adversary holds fewer features. This may be because when the adversary possesses fewer features, the effectiveness of pseudo-label inference via K-means clustering is less reliable compared to scenarios where the adversary holds more features. In such cases, selecting a larger *num_target* may result in the target class sample set including many non target class samples, thereby interfering with gradient replacement and adversely affecting the learning of the backdoor task, ultimately reducing attack effectiveness.

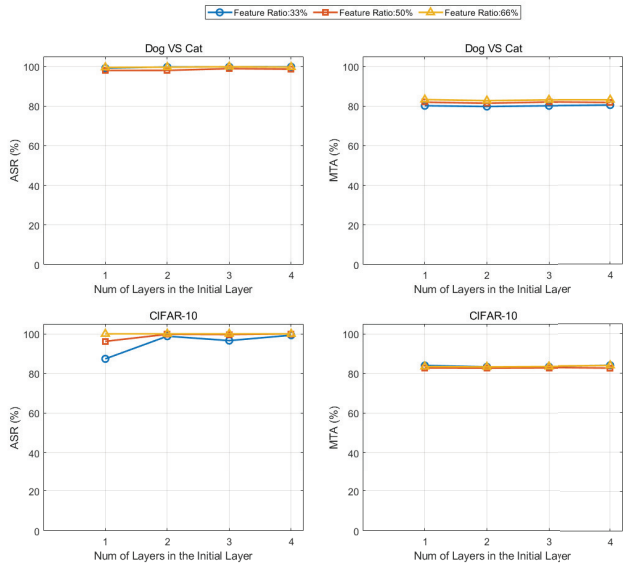


Fig. 9. Atack performance of BAEVFL under different num of layers in the Initial Layer. When the adversary possesses fewer features, an overly simplistic Initial Layer may weaken the ASR of BAEVFL to some extent.

E. Number of Layers in the Initial Layer

The Initial Layer, acting as a surrogate model for the top model, is used to fit the embeddings and their pseudo-labels obtained through the pseudo-label inference. It is then combined with the adversary's bottom model to optimize the trigger. The number of layers in the Initial Layer may impact its ability to fit the embeddings and corresponding pseudo-labels, thereby affecting the quality of the optimized trigger and, in turn, the success rate of the backdoor attack. To explore this effect, we configured the Initial Layer with "1, 2, 3, and 4" layers and conducted experiments. The results in Fig. 9 indicate that the number of layers in the Initial Layer has a minimal impact on the attack performance for the

Authorized licensed use limited to: Anhui University. Downloaded on September 07,2026 at 00:37:36 UTC from IEEE Xplore. Restrictions apply.

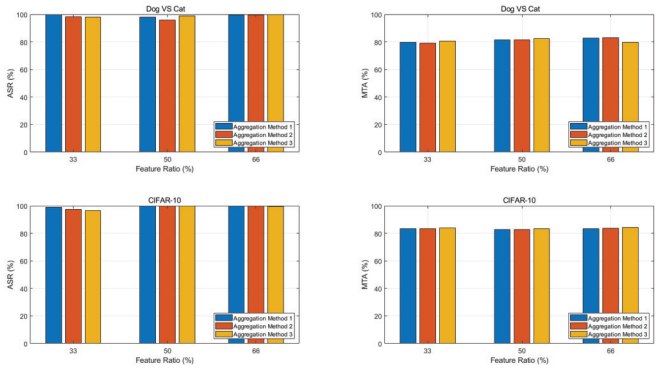


Fig. 10. Attack performance of BAEVFL under different VFL aggregation methods. Aggregation Method 1 indicate concatenation, Aggregation Method 2 indicate element-wise summation, Aggregation Method 3 indicate element- wise averaging.

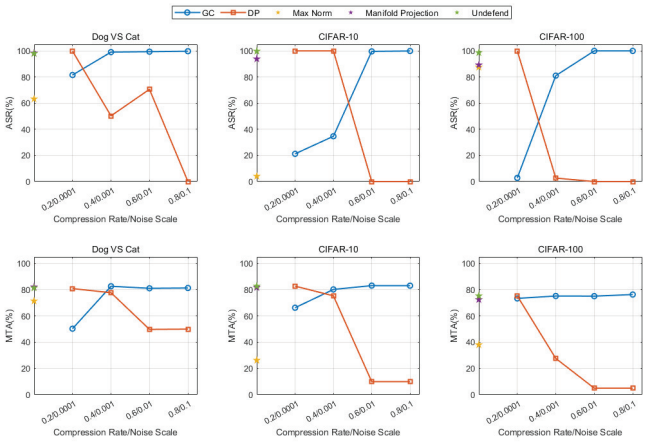


Fig. 11. Defence Evaluation on the BAEVFL. The lines labeled DP and GC display the protection performance against BAEVFL with various noise scales (0.1, 0.01, 0.001, 0.0001) and compression rates (0.8, 0.6, 0.4, 0.2), respectively. Meanwhile, the protection performances of the Manifold Projection and Max Norm methods are depicted as single points.

Dog vs. Cat dataset. However, for the CIFAR-10 dataset, overly simplistic Initial Layer can lead to a noticeable reduction in the attack success rate. For instance, when the adversary holds 33% of the feature share, the ASR is 87.39% with a single-layer Initial Layer. Increasing the Initial Layer to two layers significantly enhances the ASR to 98.81%. This may be because a simplistic Initial Layer fails to effectively learn the relationship between embeddings and their corresponding pseudo-labels, providing inadequate surrogate performance for the top model. Combining such an Initial Layer with the bottom model for trigger optimization results in suboptimal backdoor attack performance.

F. Vertical Federated Aggregation Methods

Our default experimental aggregation method is concatenation, where the active party concatenates all embeddings for the same sample as input to the top model. However, as noted in [6], the active party may also choose element-wise summation or element-wise averaging of embeddings as aggregation methods. To ensure BAEVFL's adaptability to different aggregation methods, we included these alternative aggregation methods in our experiments. The results in Fig. 10 show that BAEVFL achieves strong attack performance across all three aggregation methods, with minimal impact on model utility. These results further validate BAEVFL's robustness when facing different aggregation methods.

VII. Defense

In this section, we evaluate four potential defense methods to investigate whether current representative defense methods can effectively mitigate the proposed BAEVFL and provide recommendations for improving these methods. The four defense strategies include: 1) Differential Privacy, 2) Gradient Compression, 3) Max Norm, 4) Manifold Projection. Our defense experiments are conducted in a two-party setup, with configurations consistent with those used in the evaluation experiments.

A. Differential Privacy

As noted in [12], active parties may mitigate label privacy leakage by adding noise to gradients transmitted to participants. This may cause the adversary's bottom model to retain less label information, thereby impacting the accuracy of pseudo-label inference and further reducing the attack success rate of BAEVFL. Here, we employ Laplace noise to perturb gradients sent to each collaborator, experimenting with noise scales of 0.0001, 0.001, 0.01, and 0.1 to balance defense efficacy and model utility loss. A larger noise scale indicates stronger perturbation to the gradients, corresponding to a higher level of defense.

The results in Fig. 11 reveal that, on the Dog vs. Cat dataset, small noise levels (scales of 0.0001) fail to suppress BAEVFL effectively. Increasing the noise level (scales of 0.001) reduces the attack success rate (ASR) to 50.28%, but the model utility drops from the initial 81.43% to 77.74%. Further increasing the noise level (scales of 0.01) leads to more severe utility degradation and an unexpected rise in ASR to 70.60%. This increase in ASR may stem from class imbalance in the model after training, causing the model to predict all samples as the target label, which, in some repeated experiments, results in a 100% ASR. When the noise scales is set to 0.1, the loss values during training become NaN, causing embedding elements to also contain NaN values. This prevents the pseudo-label inference stage of BAEVFL from functioning correctly, reducing the ASR to 0%. On the CIFAR-10 dataset, when the noise scales is small (0.0001 or 0.001), DP fails to affect the attack success rate (ASR). As the noise scales increases to 0.01 or 0.1, the training process results in NaN loss values, and the trained model's accuracy drops to 10%, equivalent to random guessing. Consequently, the adversary cannot execute the pseudo-label inference phase of BAEVFL, leading to an ASR of 0%. The experimental results on the CIFAR-100 dataset are similar to those on CIFAR-10. Small noise levels fail to suppress BAEVFL effectively, while larger noise levels reduce the ASR but significantly degrade the model utility.

Thus, DP is not an ideal defense against BAEVFL, as it cannot balance defense effectiveness and model utility. Future research may focus on developing more adaptive differential privacy mechanisms, such as dynamically adjusting the noise

Authorized licensed use limited to: Anhui University. Downloaded on September 07,2026 at 00:37:36 UTC from IEEE Xplore. Restrictions apply.

scale for each training round or client based on specific training conditions. This approach could better balance backdoor security and model utility.

B. Gradient Compression

The core idea of gradient compression [12], [31] is to protect the label information held by the active party by sharing fewer gradients with participants, it may also impacting the accuracy of pseudo-label inference. Specifically, the active party transmits only the gradients with the largest absolute values to each collaborator during each training round. We tested gradient compression rates of 0.2, 0.4, 0.6, and 0.8 to investigate their impact on the defense effectiveness against BAEVFL. We define the Compression Rate as the proportion of gradients retained during the gradient compression process. Thus, a lower Compression Rate indicates a greater degree of gradient compression, corresponding to a stronger level of defense.

From the results in Fig. 11, it is evident that gradient compression does not achieve an ideal trade-off between defense effectiveness and model utility across the three datasets. On the Dog vs. Cat dataset, at a compression rate of 0.2, the model utility drops to 21.23%, while BAEVFL's ASR remains as high as 81.56%. On the CIFAR-10 dataset, when the compression rate is set to 0.4, BAEVFL's attack success rate is reduced to 34.67%, while the model accuracy drops from 82.73% (without defense) to 80.23%. When the compression rate is further reduced to 0.2, the model accuracy decreases to 66.31%. On the CIFAR-100 dataset, with a compression rate of 0.4, the model accuracy drops slightly from 75.35% (without defense) to 75.23%, while the attack success rate remains high at 81.06%. When the compression rate is reduced further to 0.2, the model accuracy declines drastically to an unacceptable 2.79%.

Overall, GC demonstrates a certain degree of defensive effectiveness against BAEVFL, but the trade-off between defense effectiveness and model utility remains unfavorable. Future research could focus on incorporating additional compression criteria beyond just the absolute gradient values to select which gradients should be clipped, as larger gradients do not necessarily contain more privacy-sensitive information.

C. Max Norm

Similar to DP and Gradient Compression, Max Norm [15] aims to mitigate label information leakage by processing the gradients transmitted from the active party to the collaborators, this may also cause the adversary's bottom model to retain less label information. Max Norm is essentially an enhanced version of Gaussian noise, applying constraints on the magnitude and direction of the gradient.

From the experimental results shown in Fig. 11, Max Norm achieves significant defense performance on the Dog vs. Cat and CIFAR-10 datasets. On the Dog vs. Cat dataset, the attack success rate drops from 98.00% to 63.26%, while on the CIFAR-10 dataset, the attack success rate decreases from 99.78% to 4.10%. However, the model accuracy also drops by 10.01% on Dog vs. Cat and 56.45% on CIFAR-10. On the CIFAR-100 dataset, BAEVFL under Max Norm defense still maintains 87.58% attack performance, while the model utility decreases from 75.35% to 38.10%.

From the experiment results, it is clear that Max Norm is not an effective defense method against BAEVFL, as its defense effectiveness comes at the cost of severe model utility degradation. Additionally, its performance on the CIFAR-100 dataset is unacceptable. Future research could explore theoretical guarantees for the privacy protection offered by Max Norm and treat the loss of model utility as another critical evaluation metric for its defense effectiveness.

D. Manifold Projection

Manifold Projection [35], [36] is another type of defense that the active party might employ. This method uses an autoencoder to clean the embeddings transmitted by each collaborating party, aiming to mitigate both backdoor and adversarial attacks.

From the experimental results shown in Fig. 11, it is evident that the Manifold Projection is ineffective on the Dog vs. Cat dataset. However, on the CIFAR-10 and CIFAR-100 datasets, it provides a certain degree of defensive effectiveness, albeit with a slight impact on model accuracy. On the CIFAR-10 dataset, applying Manifold Projection reduces the attack success rate of BAEVFL from 99.78% to 93.85%, with model accuracy dropping from 82.73% to 81.74%. On the CIFAR-100 dataset, the BAEVFL attack success rate decreases from 98.71% to 89.29%, and model accuracy drops from 75.35% to 72.49%.

Clearly, the embedding cleaning effect of Manifold Projection is insufficient to defend against BAEVFL effectively. Future research could focus on optimizing the autoencoder architecture, designing a more suitable model that can better capture the distribution of sample embeddings, and restoring poisoned sample embeddings to their original benign form, thus providing more robust defense against BAEVFL.

Summary: We evaluate four mainstream defense methods, and the experimental results and analysis reveal that current defenses are either ineffective against our proposed BAEVFL or achieve defense at the cost of significantly compromising model utility. This highlights the critical security challenges faced by the VFL paradigm and underscores the urgency of developing more targeted defense strategies. Future defense efforts could focus on reducing the sensitive information retained by the adversary's local model.

VIII. Related Work

A. Privacy Leakage in Vertical Federated Learning

Previous studies have demonstrated that VFL is prone to privacy leakage in various aspects. [12] revealed that VFL is vulnerable to label inference attacks initiated by passive parties, whether honest-but-curious or malicious. These attacks enable the passive party to infer the labels of the samples held by the active party, severely infringing upon the privacy and interests of the active party. Reference [27] have proposed data reconstruction attacks targeting VFL by optimizing gradient matching. Additionally, [32] have introduced feature inference attacks based on solving equations in VFL prediction, as well

Authorized licensed use limited to: Anhui University. Downloaded on September 07,2026 at 00:37:36 UTC from IEEE Xplore. Restrictions apply.

as feature inference attacks using Generative Adversarial Networks (GANs). Reference [33] proposed a feature inference attack aimed at the Artificial Internet of Things based VFL prediction using zero-order gradient estimation. A relation inference attack against VFL graph has also been proposed by [34].

B. Backdoor Attacks in Vertical Federated Learning

Reference [24] were the first to propose a gradient substitution-based backdoor attack for VFL. Later, [11] introduced a batch-level backdoor attack targeting VFL without a top model. However, neither of them is applicable to the widely used split neural network framework in VFL.

Other works have proposed a series of backdoor attacks for VFL based on split neural networks. These methods [9], [10] rely on label inference attacks, as introduced by [12], which require the adversary to have access to a sufficient number of samples from each class in order to construct a high-quality inference head capable of accurately predicting the class of each sample. In practice, this requirement is difficult to satisfy. Additionally, some methods [5], [6] exploit the strong similarity in gradients of samples from the same class. By combining auxiliary target class samples, these methods infer additional target class samples for poisoning. However, these backdoor attacks often incorporate label inference techniques based on sample-leve plaintext gradients that can be bypassed by encryption protections, applied during the learning process. Furthermore, these approaches typically focus on poisoning target class samples, which can lead to challenges like insufficient poisoned samples or difficulty in applying the attack to multi-class, complex tasks. In the backdoor attack on VFL under information-limited settings proposed by [8], the adversary optimizes trigger perturbations using a minimal number of known target class sample gradients during training. However, this approach can be easily thwarted by encryption protections. In contrast, [7] assumes the adversary has access to a substantial number of target class samples, leveraging these samples to optimize backdoor embedding. These optimized backdoor embedding are then used to replace the embeddings of the known target class samples. The BAEVFL proposed in this paper differs from prior work. It allows the adversary to execute a powerful backdoor attack using only a minimal number of target class samples without sample-level plaintext gradients. The attack is especially applicable to encryption-protected VFL, which is a more realistic attack scenario. It exposes the significant security vulnerabilities in this paradigm.

IX. Conclusion

In this paper, we propose a novel backdoor attack targeting encryption-protected VFL, where the attacker only needs a very small number of target class samples to successfully launch a powerful and stealthy backdoor attack. To the best of our knowledge, this is the first work explore backdoor attacks in encryption-protected VFL based on split neural networks. We validated the robustness of the proposed attack through extensive experiments, covering different aggregation methods, varying levels of complexity in top models, and scenarios with multiple participants. Ablation studies were conducted to explore potential influencing factors. Our work exposes the significant security threats inherent in the encryption-protected VFL paradigm. Furthermore, through subsequent defense experiments, we demonstrate that current defense methods are ineffective in mitigating our attack and provide key recommendations for improving these defenses.

Acknowledgment

The authors are very grateful to the anonymous referees for their detailed comments and suggestions regarding this article.

References

[1] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. Y. Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proc. 20th Int. Conf. Artif. Intell. Statist.*, vol. 54, 2017, pp. 1273–1282.

[2] Q. Yang, Y. Liu, T. Chen, and Y. Tong, "Federated machine learning: Concept and applications," 2019, *arXiv:1902.04885*.

[3] S. Saha and T. Ahmad, "Federated transfer learning: Concept and applications," *Intelligenza Artificiale*, vol. 15, no. 1, pp. 35–44, 2021.

[4] D. Romanini et al., "PyVertical: A vertical federated learning framework for multi-headed SplitNN," 2021, *arXiv:2104.00489*.

[5] Y. Xuan, X. Chen, Z. Zhao, B. Tang, and D. Ye, "Practical and general backdoor attacks against vertical federated learning," in *Proc. Joint Eur. Conf. Mach. Learn. Knowl. Discov. Databases (ECML-PKDD)*. Cham, Switzerland: Springer, Jan. 2023, pp. 402–417.

[6] Y. Bai, Y. Chen, H. Zhang, W. Xu, H. Weng, and D. Goodman, "VILLAIN: Backdoor attacks against vertical split learning," in *Proc. 32nd USENIX Secur. Symp.*, 2023, pp. 2743–2760.

[7] Y. He et al., "Backdoor attack against split neural network-based vertical federated learning," *IEEE Trans. Inf. Forensics Security*, vol. 19, pp. 748–763, 2024.

[8] P. Chen, J. Yang, J. Lin, Z. Lu, Q. Duan, and H. Chai, "A practical clean-label backdoor attack with limited information in vertical federated learning," in *Proc. IEEE Int. Conf. Data Mining (ICDM)*, Shanghai, China, Dec. 2023, pp. 41–50.

[9] M. Naseri, Y. Han, and E. De Cristofaro, "BadVFL: Backdoor attacks in vertical federated learning," in *Proc. IEEE Symp. Secur. Privacy (SP)*, San Francisco, CA, USA, May 2024, pp. 2013–2028.

[10] Y. Gu and Y. Bai, "LR-BA: Backdoor attack against vertical federated learning using local latent representations," *Comput. Secur.*, vol. 129, Jun. 2023, Art. no. 103193.

[11] T. Zou et al., "Defending batch-level label inference and replacement attacks in vertical federated learning," *IEEE Trans. Big Data*, vol. 10, no. 6, pp. 1016–1027, Nov./Dec. 2024.

[12] C. Fu et al., "Label inference attacks against vertical federated learning," in *Proc. 31st USENIX Security Symp. (USENIX Security)*, 2022, pp. 1397–1414.

[13] X. Yang, J. Sun, Y. Yao, J. Xie, and C. Wang, "Differentially private label protection in split learning," 2022, *arXiv:2203.02073*.

[14] D. Pasquini, G. Ateniese, and M. Bernaschi, "Unleashing the tiger: Inference attacks on split learning," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Nov. 2021, pp. 2113–2129.

[15] O. Li et al., "Label leakage and protection in two-party split learning," 2021, *arXiv:2102.08504*.

[16] J. Sun, X. Yang, Y. Yao, and C. Wang, "Label leakage and protection from forward embedding in vertical federated learning," 2022, *arXiv:2203.01451*.

[17] Q. Zhang, C. Wang, H. Wu, C. Xin, and T. V. Phuong, "GELU-Net: A globally encrypted, locally unencrypted deep neural network for privacy-preserved learning," in *Proc. 27th Int. Joint Conf. Artif. Intell.*, Jul. 2018, pp. 3933–3939.

[18] Y. Zhang and H. Zhu, "Additively homomorphical encryption based deep neural network for asymmetrically collaborative machine learning," 2020, *arXiv:2007.06849*.

[19] Y. Liu et al., "Trojaning attack on neural networks," in *Proc. 25th Annu. Netw. Distrib. System Secur. Symp. (NDSS)*, 2018, pp. 1–16.

[20] A. Shafahi et al., "Poison frogs! Targeted clean-label poisoning attacks on neural networks," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 31, Dec. 2018, pp. 6106–6116.

Authorized licensed use limited to: Anhui University. Downloaded on September 07,2026 at 00:37:36 UTC from IEEE Xplore. Restrictions apply.

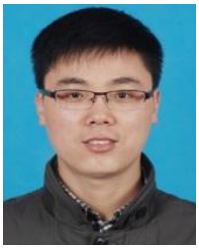
[21] T. Gu, K. Liu, B. Dolan-Gavitt, and S. Garg, "BadNets: Evaluating backdooring attacks on deep neural networks," *IEEE Access*, vol. 7, pp. 47230–47244, 2019.
[22] L. Truong et al., "Systematic evaluation of backdoor data poisoning attacks on image classifiers," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. Workshops*, Jun. 2020, pp. 788–789.
[23] C. Xie, K. Huang, P. Chen, and B. Li, "DBA: Distributed backdoor attacks against federated learning," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, Apr. 2020.
[24] Y. Liu, Z. Yi, and T. Chen, "Backdoor attacks and defenses in feature-partitioned collaborative learning," 2020, *arXiv:2007.03608*.
[25] A. Madry, A. Makelov, L. Schmidt, D. Tsipras, and A. Vladu, "Towards deep learning models resistant to adversarial attacks," 2017, *arXiv:1706.06083*.
[26] *FATE*. Accessed: Nov. 2024. [Online]. Available: https://fate.readthedocs.io
[27] X. Jin, P.-Y. Chen, C.-Y. Hsu, C.-M. Yu, and T. Chen, "CAFE: Catastrophic data leakage in vertical federated learning," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 34, 2021, pp. 994–1006.
[28] Y. Liu et al., "Vertical federated learning: Concepts, advances, and challenges," *IEEE Trans. Knowl. Data Eng.*, vol. 36, no. 7, pp. 3615–3634, Jul. 2024.
[29] C. E. Joern and D. Kiselbach, "New consumer product safety laws in Canada and the United States: Bus. on the border," *Global Trade Customs J.*, vol. 7, no. 1, pp. 1–6, Jan. 2012.
[30] P. Voigt and A. von dem Bussche, *The EU General Data Protection Regulation (GDPR): A Practical Guide*, 1st ed., Cham, Switzerland: Springer, 2017.
[31] Y. Lin, S. Han, H. Mao, Y. Wang, and W. J. Dally, "Deep gradient compression: Reducing the communication bandwidth for distributed training," 2017, *arXiv:1712.01887*.
[32] X. Luo, Y. Wu, X. Xiao, and B. C. Ooi, "Feature inference attack on model predictions in vertical federated learning," in *Proc. 37th IEEE Int. Conf. Data Eng. (ICDE)*, Chania, Greece, 2021, pp. 181–192.
[33] R. Yang, J. Ma, J. Zhang, S. Kumari, S. Kumar, and J. J. P. C. Rodrigues, "Practical feature inference attack in vertical federated learning during prediction in artificial Internet of Things," *IEEE Internet Things J.*, vol. 11, no. 1, pp. 5–16, Jan. 2024.
[34] P. Qiu, "Your labels are selling you out: Relation leaks in vertical federated learning," *IEEE Trans. Depend. Sec. Comput.*, vol. 20, no. 5, pp. 3653–3668, Sep. 2023.
[35] D. Meng and H. Chen, "MagNet: A two-pronged defense against adversarial examples," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Oct. 2017, pp. 135–147.
[36] B. Lindqvist, S. Sugrim, and R. Izmailov, "AutoGAN: Robust classifier against adversarial attacks," 2018, *arXiv:1812.03405*.



Junyu Chen received the B.S. degree from Changsha University in 2022. He is currently pursuing the Ph.D. degree with the Department of Computer Science and Technology, Anhui University. His current research interests include trustworthy artificial intelligence.



Depeng Chen received the Ph.D. degree in computer science from the Universitat Autónoma de Barcelona in 2020. He is currently a Lecturer at the School of Computer Science and Technology, Anhui University. His research interests include trustworthy artificial intelligence systems, communication anonymity, and privacy.



Jie Cui (Senior Member, IEEE) was born in Henan, China, in 1980. He received the Ph.D. degree from the University of Science and Technology of China in 2012. He is currently a Professor and the Ph.D. Supervisor of the School of Computer Science and Technology, Anhui University. He has more than 200 scientific publications in reputable journals (e.g., IEEE TRANSACTIONS ON DEPENDABLE AND SECURE COMPUTING, IEEE TRANSACTIONS ON INFORMATION FORENSICS AND SECURITY, IEEE JOURNAL ON SELECTED AREAS IN COMMUNICATIONS, IEEE TRANSACTIONS ON MOBILE COMPUTING, IEEE TRANSACTIONS ON PARALLEL AND DISTRIBUTED SYSTEMS, and IEEE TRANSACTIONS ON COMPUTERS), academic books, and international conferences. His current research interests include applied cryptography, the IoT security, and security of vehicular ad hoc networks. He is in the Editorial Board of several international journals, such as *IET Communications* and *Security and Communication Networks*.



Hong Zhong was born in Anhui, China, in 1965. She received the Ph.D. degree in computer science from the University of Science and Technology of China in 2005. She is currently a Professor and the Ph.D. Supervisor of the School of Computer Science and Technology, Anhui University. She has more than 200 scientific publications in reputable journals (e.g., IEEE JOURNAL ON SELECTED AREAS IN COMMUNICATIONS, IEEE TRANSACTIONS ON PARALLEL AND DISTRIBUTED SYSTEMS, IEEE TRANSACTIONS ON MOBILE COMPUTING, IEEE TRANSACTIONS ON DEPENDABLE AND SECURE COMPUTING, IEEE TRANSACTIONS ON INFORMATION FORENSICS AND SECURITY, IEEE TRANSACTIONS ON INTELLIGENT TRANSPORTATION SYSTEMS, IEEE TRANSACTIONS ON MULTIMEDIA, IEEE TRANSACTIONS ON VEHICULAR TECHNOLOGY, IEEE TRANSACTIONS ON NETWORK AND SERVICE MANAGEMENT, IEEE TRANSACTIONS ON CLOUD COMPUTING, IEEE TRANSACTIONS ON INDUSTRIAL INFORMATICS, IEEE TRANSACTIONS ON INDUSTRIAL ELECTRONICS, and IEEE TRANSACTIONS ON BIG DATA), academic books, and international conferences. Her research interests include applied cryptography, the IoT security, vehicular ad hoc networks, cloud computing security, and software-defined networking (SDN).

Authorized licensed use limited to: Anhui University. Downloaded on September 07,2026 at 00:37:36 UTC from IEEE Xplore. Restrictions apply.